Entregável 1 explicado

Rede de distribuição: histórico de períodos

Algoritmos e Estruturas de Dados – BCC – 2026.2

Laura da Costa Corte Cordeiro e Ana Helena de Souza Macedo

O que este documento cobre

- Os assuntos da disciplina cobrados no Entregável 1 e a teoria de cada um.
- A lógica do arquivo `e1.py`, função por função, com os trechos de código do próprio arquivo.
- Uma legenda com todas as notações usadas no código, na documentação e neste texto.
- Uma simulação completa do exemplo do enunciado, as complexidades e as perguntas prováveis da entrevista.

Sumário

- 1. Legenda das notações
- 2. O que o Entregável 1 pede
- 3. Assuntos que caem
- 4. Lógica do código
- 5. Simulação do exemplo
- 6. Complexidades
- 7. Armadilhas comuns
- 8. Perguntas prováveis da entrevista

1. Legenda das notações

Notações do enunciado, usadas nas complexidades e nas docstrings:

Símbolo	Significado
n	Número de depósitos vivos no momento
P	Número de períodos (no enunciado, também é o total de lotes lido da entrada)
h	Altura da pilha de prosperidade (quantos elementos ela tem)
k	Número do período (k = 1 é logo após o setup; k = 2, 3, ... são os lotes)
L_k	Lucro total do período k: soma dos lucros dos depósitos vivos ao fim do lote k
T	Tolerância mínima de lucro (lida no E1, mas só usada a partir do E2)
q	Quantidade de operações de um lote
[i, j], S	Intervalo de períodos i até j e a sua soma $L_i + ... + L_j$ (resposta do MELHOR)
(j, r)	Elemento da pilha: j é o período do ganho e r é o resíduo (parte ainda não consumida)
h b s i r	Resposta do PROSPERIDADE: altura, período da base, soma dos resíduos, período e resíduo do topo

Nomes usados no código:

Nome	O que guarda
`tokens`	Toda a entrada, quebrada em palavras
`pos`	Índice do próximo token ainda não lido
`lucros`	Dicionário id : lucro atual de cada depósito vivo
`total`	Soma dos lucros dos vivos (vira L_k ao fim do lote)
`kadane`	Dicionário com o estado da soma máxima de subvetor consecutivo
`fim_soma, fim_ini`	Soma e início do melhor intervalo que termina no último período
`mi, mj, ms`	Melhor intervalo encontrado até agora, [mi, mj], e a sua soma ms
`pilha, soma_pilha`	Pilha de prosperidade (lista de pares [j, r]) e a soma dos resíduos
`d`	Prejuízo que ainda falta descontar da pilha (d = −L)
`saida`	Lista das respostas, impressas todas juntas no final

Notações de complexidade:

Termo	Significado
O(f)	O custo cresce no máximo proporcionalmente a f
Pior caso	Vale para toda execução da operação, sem exceção
Esperado	Média sobre a aleatoriedade (ex.: como as chaves se espalham na tabela hash); com azar, uma operação pode demorar mais
Amortizado	Média sobre uma sequência de operações, sem probabilidade: algumas são caras, mas o total é limitado

2. O que o Entregável 1 pede

Operações exercitadas: **INSERIR**, **LUCRO**, **MELHOR** e **PROSPERIDADE**. As rotas são lidas, mas descartadas. O enunciado espera:

- um registro dos depósitos com acesso por identificador, com o lucro total do período mantido **sem varrer todos os depósitos**;
- a pilha de prosperidade, com as regras do enunciado;
- a resposta de MELHOR **sem reexaminar todo o histórico** a cada consulta;
- a justificativa, na documentação, da estrutura escolhida para o registro.

3. Assuntos que caem

O enunciado lista como tópicos do E1: **pilha, análise amortizada, programação dinâmica (soma máxima de subvetor consecutivo) e busca por chave**.

3.1 Busca por chave: tabela hash

Uma tabela hash guarda pares chave : valor. Uma *função de hash* transforma a chave numa posição de um vetor interno, e é lá que o valor fica. Duas chaves podem cair na mesma posição (*colisão*); o Python resolve isso por endereçamento aberto e aumenta a tabela quando ela enche.

Estrutura	Busca por id	Por que não (ou sim)
Vetor indexado pelo id	O(1)	Os ids vão até 10^9: seriam bilhões de posições
Lista de pares	O(n)	Cada LUCRO percorreria até 5000 depósitos
Árvore balanceada	O(log n)	Mantém ordem, que não é necessária aqui
Tabela hash (dicionário)	**O(1) esperado**	É a escolha: rápida e com memória proporcional a n

O custo é O(1) **esperado** (e não pior caso) porque, se muitas chaves colidirem, uma busca pode percorrer vários elementos.

3.2 Manutenção incremental e invariantes

Em vez de recalcular um valor do zero, ele é ajustado a cada mudança. Um **invariante** é uma propriedade que vale entre todas as operações; é ele que garante que o valor mantido está certo. No E1: *total == soma dos lucros do dicionário*.

3.3 Programação dinâmica: soma máxima de subvetor consecutivo

Programação dinâmica resolve um problema combinando respostas de subproblemas menores, sem recalculá-los. Aqui, o subproblema é **E(k) = maior soma de um intervalo que termina exatamente em k**. Ou o intervalo é só o período k, ou é o melhor que terminava em k − 1 estendido:

$$\mathbf{E(k) = L_k + \max(E(k-1), 0)}$$

A resposta é o maior E(k) já visto. Como cada passo só depende do anterior, basta guardar dois estados e atualizá-los a cada período: esse é o **algoritmo de Kadane**, na versão que é atualizada a cada novo dado.

Abordagem	Custo por MELHOR
Testar todos os intervalos	$O(P^2)$
Rodar o Kadane do zero a cada consulta	O(P)

Abordagem	Custo por MELHOR
Kadane atualizado a cada período (o do código)	**O(1)**

3.4 Pilha

Estrutura **LIFO** (o último a entrar é o primeiro a sair): só se mexe no topo. Em Python, uma lista serve de pilha: `append` e `pop` no fim custam O(1) amortizado. A pilha de prosperidade é LIFO porque um prejuízo consome primeiro o ganho mais recente.

3.5 Análise amortizada

Mede o custo médio por operação ao longo de uma sequência, mesmo que algumas operações isoladas sejam caras. Dois métodos:

- **Agregado (contagem):** soma o custo total de todas as operações e divide pelo número delas.
- **Potencial:** define uma função Φ do estado; custo amortizado = custo real + variação de Φ. Operações baratas "guardam crédito" que as caras gastam.

Ela é usada para justificar o O(1) amortizado da pilha (seção 4.5).

4. Lógica do código

4.1 Visão geral do fluxo

```
ler setup (n depósitos; m rotas puladas)  ->  registrar período 1
para cada um dos P lotes:
    ler q
    executar as q operações (MELHOR/PROSPERIDADE enxergam até k-1)
    k += 1; registrar período k (Kadane e pilha com L_k = total)
imprimir todas as respostas
```

4.2 Leitura da entrada

A entrada inteira é lida de uma vez e quebrada em tokens. O índice `pos` avança a cada valor lido, o que torna o programa independente das quebras de linha.

```
tokens = sys.stdin.read().split()
n, m, T = int(tokens[0]), int(tokens[1]), int(tokens[2])
pos = 3

lucros = {}
total = 0
for _ in range(n):
    ident, x, y, lucro = (int(t) for t in tokens[pos:pos + 4])
    lucros[ident] = lucro
    total += lucro
    pos += 4
pos += 3*m
```

Operação	Tokens consumidos depois do nome
INSERIR id x y L r	5 + 3·r (as r rotas vêm logo depois e não contam em q)
LUCRO id L	2
MELHOR, PROSPERIDADE	0

4.3 Registro e total incremental

```
if op == "INSERIR":
    ident, x, y, lucro, r = (int(t) for t in tokens[pos:pos + 5])
    pos += 5 + 3 * r
    lucros[ident] = lucro
    total += lucro

elif op == "LUCRO":
    ident, lucro = int(tokens[pos]), int(tokens[pos+1])
    pos += 2
    total += lucro - lucros[ident]
    lucros[ident] = lucro
```

- **INSERIR:** grava o lucro e soma ao total.
- **LUCRO:** substitui (não soma). Por isso o total é ajustado pela diferença (novo − antigo), **antes** de gravar o novo valor; se gravasse antes, a diferença daria 0.

4.4 Kadane: novo_kadane, atualizar_kadane e melhor

```
def novo_kadane():
    return {"fim_soma" : 0, "fim_ini": 0, "mi" : 0, "mj" : 0, "ms" : 0}
```

```
def atualizar_kadane(kadane, k, L):
    if k > 1 and kadane["fim_soma"] >= 0:
        kadane["fim_soma"] += L
    else:
        kadane["fim_soma"] = L
        kadane["fim_ini"] = k

    s, i = kadane["fim_soma"], kadane["fim_ini"]
    if (k == 1 or s > kadane["ms"] or (s == kadane["ms"] and (i, k) < (kadane["mi"], kadane["mj"]))):
        kadane["mi"], kadane["mj"], kadane["ms"] = i, k, s
```

```
def melhor(kadane):
    return f"{kadane["mi"]} {kadane["mj"]} {kadane["ms"]}"
```

- **Estende com >= 0 (e não > 0):** se a soma anterior é 0, estender ou recomeçar dá a mesma soma, mas estender mantém o início mais antigo, que é o desempate pedido (menor i).
- **Comparação com o melhor global:** troca se a soma for maior, ou igual com (i, j) menor. Em Python, `(i, k) < (mi, mj)` compara tuplas exatamente assim.
- **k == 1** inicializa o melhor global, que ainda não existe.
- **Sempre não vazio:** com todos os L negativos, o estado recomeça a cada período, então o resultado é o maior período isolado.
- **MELHOR só lê** o estado: O(1).

4.5 Pilha de prosperidade: atualizar_pilha e prosperidade

```
def atualizar_pilha(pilha, soma, k, L):
    if L > 0:
        pilha.append([k, L])
        soma += L
    elif L < 0:
        d = -L
        while d > 0 and pilha:
            j, r = pilha[-1]
            if r > d:
                pilha[-1][1] = r - d
                soma -= d
                d = 0
            else:
                pilha.pop()
                soma -= r
                d -= r
    return soma
```

```
def prosperidade(pilha, soma):
    if not pilha:
        return "0"
    b = pilha[0][0]
    i, r = pilha[-1]

    return f"{len(pilha)} {b} {soma} {i} {r}"
```

- **L > 0:** empilha [k, L]. **L = 0:** nada muda. **L < 0:** o prejuízo d = −L consome a pilha a partir do topo.
- **r > d (e não r >= d):** com r = d o topo deve sair; com >= ficaria um resíduo 0, e os resíduos devem ser positivos.
- O topo reduzido **mantém o j original** (o período em que o ganho aconteceu).
- Os pares são listas [j, r], e não tuplas, para que o resíduo do topo possa ser alterado no lugar.
- Se a pilha esvaziar e sobrar prejuízo, a sobra é descartada.

Por que é O(1) amortizado se existe um while

• Agregado: cada elemento é empilhado uma vez e desempilhado no máximo uma vez. Em P períodos há no máximo P pushes, logo no máximo P pops no total: O(P) no total, O(1) por período.

• Potencial: Φ = altura da pilha. Um push custa 1 e aumenta Φ em 1 (amortizado 2). Desempilhar t elementos custa t + 1 e diminui Φ em t (amortizado 1).

4.6 Registro dos períodos

```
k += 1
atualizar_kadane(kadane, k, total)
soma_pilha = atualizar_pilha(pilha, soma_pilha, k, total)
```

O registro fica **depois** do laço das operações e **dentro** do laço dos lotes. Por isso: (1) acontece uma vez por lote, mesmo com q = 0; (2) MELHOR e PROSPERIDADE do lote k enxergam só até k − 1, como o enunciado pede, porque L_k ainda não foi registrado.

5. Simulação do exemplo

Lucros dos períodos no exemplo do enunciado: L = 22, −8, 15, −29, 11.

k	L_k	fim_soma, fim_ini	Melhor global	Pilha depois	soma_pilha
1	22	22, 1	1 1 22	[(1,22)]	22
2	−8	14, 1 (22 >= 0: estende)	1 1 22	[(1,14)]	14
3	15	29, 1	1 3 29	[(1,14), (3,15)]	29
4	−29	0, 1	1 3 29	[ ] (15 e 14 consumidos)	0
5	11	11, 1 (0 >= 0: estende)	1 3 29	[(5,11)]	11

Período 4: d = 29. Topo (3, 15): 15 <= 29, sai, d = 14. Topo (1, 14): 14 <= 14, sai, d = 0. A pilha fica vazia, sem sobra.

Lote	Consulta	Saída	Por quê
2	PROSPERIDADE	1 1 22 1 22	Pilha depois do período 1
2	MELHOR	1 1 22	Só existe o período 1
3	PROSPERIDADE	1 1 14 1 14	Pilha [(1,14)]
4	PROSPERIDADE	2 1 29 3 15	Altura 2, base 1, soma 29, topo (3,15)
5	PROSPERIDADE	0	Pilha vazia depois do período 4
5	MELHOR	1 3 29	22 − 8 + 15 = 29
6	PROSPERIDADE	1 5 11 5 11	Pilha [(5,11)]
6	MELHOR	1 3 29	O melhor histórico continua sendo [1, 3]

6. Complexidades

Operação	Tempo	Justificativa
INSERIR	O(1) esperado	Uma inserção no dicionário e uma soma (as rotas são só puladas)
LUCRO	O(1) esperado	Uma busca e uma atribuição no dicionário
Registrar período	O(1) amortizado	Kadane é constante; a pilha é amortizada
MELHOR	O(1)	Lê o estado do Kadane
PROSPERIDADE	O(1)	len, base e topo por índice; a soma já está mantida
Programa inteiro	O(tamanho da entrada)	Cada token é processado um número constante de vezes

Espaço: O(n) para o dicionário, até O(P) para a pilha e O(1) para o Kadane, além da entrada e da saída.

7. Armadilhas comuns

Erros que derrubam o E1

- Pular só 5 tokens no INSERIR: as rotas passam a ser lidas como operações.
- Gravar o lucro novo antes de ajustar o total no LUCRO.
- Registrar o período antes do lote, duas vezes, ou esquecer quando q = 0.
- Usar > 0 no Kadane (perde o desempate pelo menor i) ou r >= d na pilha (deixa resíduo 0).
- Imprimir mensagens de depuração na saída padrão.
- Aspas duplas dentro de uma f-string com aspas duplas: só funciona a partir do Python 3.12.

8. Perguntas prováveis da entrevista

Pergunta	Ideia central da resposta
Por que dicionário e não vetor ou lista?	Ids até 10^9 impedem vetor; lista é O(n) por busca; hash dá O(1) esperado; árvore seria O(log n) sem precisar de ordem.
Como o total é mantido sem varrer?	Ajuste a cada mudança: += L no INSERIR, += (novo − antigo) no LUCRO. Invariante: total = soma dos lucros.
Por que a pilha é O(1) amortizado com um while?	Cada elemento entra uma vez e sai no máximo uma vez; o total de pops é limitado pelo total de pushes.
Como MELHOR evita reler o histórico?	Kadane: guarda o melhor que termina no último período e o melhor global.
Como trata empates no MELHOR?	Estende com soma anterior >= 0 e compara (i, j) como tupla no global.
Por que a consulta do lote k vê só até k − 1?	O período k só é registrado depois do lote.
E se o desempate fosse o maior i?	Estender só com soma anterior > 0 e trocar a comparação do global para preferir i maior.
Pior caso, esperado e amortizado: diferença?	Pior caso vale sempre; esperado é média sobre aleatoriedade; amortizado é média sobre uma sequência, sem probabilidade.